Computational Visual Media
https://doi.org/10.1007/s41095-021-0251-7 Vol. 8, No. 3, September 2022, 467–481

Research Article

Robust and efficient edge-based visual odometry

Feihu Yan[1], **Zhaoxin Li**[2], **and Zhong Zhou**[1] (✉)

© The Author(s) 2021.

Abstract Visual odometry, which aims to estimate relative camera motion between sequential video frames, has been widely used in the fields of augmented reality, virtual reality, and autonomous driving. However, it is still quite challenging for state-of-the-art approaches to handle low-texture scenes. In this paper, we propose a robust and efficient visual odometry algorithm that directly utilizes edge pixels to track camera pose. In contrast to direct methods, we choose reprojection error to construct the optimization energy, which can effectively cope with illumination changes. The distance transform map built upon edge detection for each frame is used to improve tracking efficiency. A novel weighted edge alignment method together with sliding window optimization is proposed to further improve the accuracy. Experiments on public datasets show that the method is comparable to state-of-the-art methods in terms of tracking accuracy, while being faster and more robust.

Keywords visual odometry (VO); edge structure; distance transform; low-texture

1 Introduction

As one of the essential technologies in many emerging applications, such as robot navigation [1], augmented reality (AR) [2], and virtual reality (VR) [3], simultaneous localization and mapping (SLAM) has received widespread attention in recent years. With the help of various sensors, such as cameras, lidar, etc., SLAM can build a 3D model of the surrounding environment while tracking the position of the sensor. Specifically, SLAM using only cameras is referred to as visual SLAM [4–8]. Generally regarded as a component or special case of visual SLAM, visual odometry (VO) [9–13] mainly solves the basic problem of how to track camera pose in unknown environments.

Typical existing VO algorithms can be divided into two categories. The first comprises feature-based (indirect) methods [7, 14, 15], which extract corner points or other distinguishable feature points from images, and estimates camera motion by minimizing the reprojection error of matched features. The second category comprises direct methods [4, 11], which directly use image pixels to estimate camera pose by optimizing photometric error.

Benefitting from the invariance of feature descriptors, the feature-based method is robust for a variety of well-textured scenes, but the performance drops in low-texture areas since it relies highly on extracting sufficient features. Although the direct methods deal better with this low-texture problem by using more image information, the fundamental assumption on photo consistency makes them quite sensitive to illumination changes, which also affect their robustness in real applications.

To make VO systems more robust, researchers began to pay attention to high-level features, such as edge- [12, 16–20] or line- [21–32] features which are widespread in man-made scenes. Integration of edge information has been proven to improve the robustness of VO systems, but also brings extra computational overhead and has a negative effect on real-time performance.

In this paper, we propose a robust and efficient edge-based VO system, called *distance transform visual odometry* (DTVO), to mitigate the above mentioned problems. Based on the observation that edge information is abundant in man-made environments,

1 State Key Laboratory of Virtual Reality Technology and Systems, Beihang University, Beijing 100191, China. E-mail: F. Yan, yanfeihu@buaa.edu.cn; Z. Zhou, zz@buaa.edu.cn (✉).

2 Institute of Computing Technology, Chinese Academy of Sciences, Beijing 100190, China. E-mail: cszli@hotmail.com.

Manuscript received: 2021-06-30; accepted: 2021-08-13

even in homogeneous areas like wall surfaces, our method takes advantage of edge features to deal with the low-texture problem. More specifically, edge pixels are detected from each frame using the Canny algorithm [33] and utilized throughout the tracking process. In contrast to direct VO systems, we employ the geometric reprojection error instead of the photometric residual, which is more robust with respect to illumination changes. To meet the demands of real-time tracking, edges in the reference frame are projected into the distance transform (DT) map of the current frame to efficiently calculate the residuals. Moreover, a novel dynamic weighted edge registration method combined with a pyramid coarse-to-fine scheme is proposed to improve tracking accuracy. A local sliding window optimization step is also integrated to refine the depth map as well as camera pose. The proposed method is not only suitable for monocular cameras but can also be extended to RGB-D sensors. The proposed VO system is able to achieve 70 fps for 640×480 resolution images from public datasets on a CPU.

Our contributions can be summarized as

- a 3D–2D edge alignment method that effectively tracks camera pose by leveraging local geometric information in the edge-driven DT map,
- a sliding window optimization approach to refine the propagated depth map, as well as all camera poses, within the local window, and
- using the above two modules, a novel edge-based VO system that efficiently integrates edge information to more robustly extract camera pose.

To the best of our knowledge, this is the first real-time VO pipeline for monocular cameras that utilizes DT maps. Furthermore, it can be extended to RGB-D sensors.

The remainder of this paper is organized as follows. Section 2 introduces related work. Section 3 provides the problem formulation. Tracking and local mapping based on edges are presented in Sections 4 and 5, respectively. In Section 6, we provide experimental comparisons with state-of-art algorithms. Finally, conclusions and future work are discussed in Section 7.

2 Related work

In this section, we describe related work on visual SLAM (including VO) which can be broadly divided into feature-based methods, direct methods, and edge-related approaches.

2.1 Feature-based SLAM

Feature-based methods have dominated the field of visual SLAM in the past decades. Most of them are inspired by the framework of PTAM (*parallel tracking and mapping*) [34], one of the most groundbreaking SLAM systems. PTAM divides the entire system into two independent threads, a real-time camera tracking front-end, and a mapping back-end based on optimization. One of the most representative works in recent years is ORB-SLAM [7, 14], which extracts ORB features for tracking and mapping, and innovates with a three-thread architecture by adding a loop closing component. Due to its high tracking accuracy and good scalability, ORB-SLAM has become one of the most popular state-of-the-art SLAM frameworks. However, this kind of method needs to detect reliable feature points in images, preventing their application in low-textured environments.

2.2 Direct methods

In contrast to feature-based approaches, direct methods directly use whole image alignment based on photometric error to track camera pose. Omitting the steps of feature extraction and descriptor calculation makes the system more efficient. At the same time, more image information also means that a more dense map can be recovered. These advantages have increased the popularity of the direct method in recent years. Newcombe et al. [35] presented *dense tracking and mapping* (DTAM), which achieves dense and smooth depth estimation by using a non-convex optimization process. This system needs GPU acceleration to run in real time. Engel et al. [4, 9] proposed the large scale direct monocular SLAM (LSD-SLAM) system, which performs tracking and mapping directly over image pixel intensities. It is impressive that this system is able to reconstruct a semi-dense map and operate in real time without GPU acceleration. The most influential direct method recently is *direct sparse odometry* (DSO) [11], which provides state-of-the-art performance in terms of both accuracy and robustness for monocular camera tracking.

LSD-SLAM [4] and DSO [11] are quite popular amongst direct methods, and there are many extensions [5, 36–38] to these two methods. There

are also several semi-direct systems that combine the complementary strengths of direct and feature-based methods, such as SVO [10] and LCSD-SLAM [39].

The main problem of direct methods is that they rely on photometric minimization, making them sensitive to illumination changes.

2.3 Edge-related SLAM

Researchers have tried to integrate edge information into visual SLAM for a long time. Early works like Refs. [21, 22] integrated straight lines into the filter-based method, but many mismatches occur when the help of descriptors is unavailable.

Intuitively, the feature-based method is based on the extraction and matching of feature points and descriptors, which can be easily extended to edge features and descriptors. Pumarola et al. [25] proposed PL-SLAM, an extension of ORB-SLAM [14], which uses a *line segment detector* (LSD) [40] to extract linear features and calculates a *line band descriptor* (LBD) [41]. Zhang et al. [42] added line features detected by LSD [40] to ORB-SLAM [14], which provides long-term constraints using planes. Zuo et al. [26] fully took into account the parametric representation of spatial straight-lines, and introduced the orthogonal representation method to solve the problem of over-parameterization.

Incorporating line features into direct methods can also improve robustness. On the basis of direct visual odometry [9], Yang and Scherer [30] introduced LSD [40] and LBD [41] into a direct SLAM framework that can improve robustness, but the line segment detection and descriptor calculation, as well as subsequent feature matching, greatly increased computational consumption. Li et al. [31] introduced a collinear constraint into the DSO framework [11] through line segments detected by LSD [40], and reduced the computational cost by removing non-line pixels.

Gomez-Ojeda et al. [27] introduced LSD [40] into the SVO [10] framework to obtain a more robust system capable of dealing with untextured environments. They subsequently combined ORB features and line features to propose a VO method [28] and a visual SLAM method [29] for stereo cameras, which can produce rich geometrical maps, but their odometry method requires reduced image resolution to achieve real-time performance.

The above methods show that the introduction of edge features can effectively improve robustness, but extracting line features and matching corresponding descriptors are time-consuming, which precludes real-time application. Moreover, the over-parameterization problem is also a hindrance for optimization.

Instead of line segment features, some researchers try to estimate camera motion directly from the edge pixels. Wang et al. [16] presented a real-time RGB-D VO system that combines photometric error with edge distance error provided by the DT map. Kuse and Shen [17] proposed a direct RGB-D VO system which utilizes the sub-gradient method to handle non-differentiable functions. Schenk and Fraundorfer [18, 19] proposed DT-based RGB-D VO systems combining Canny [33] and machine-learned edges respectively, and then later presented RESLAM [20], which is a complete edge-based SLAM pipeline for RGB-D sensors.

Although the efficient tracking brought by DT has been demonstrated in the above methods, the depth information seems to be indispensable. In this work, we proposed a novel VO pipeline that can be used for both monocular cameras and RGB-D sensors.

3 Overview

In this section, we introduce the formulation of camera motion used in our paper and give a brief overview of our edge-based VO framework.

3.1 Notation

Similar to direct methods [4, 11], we maintain a reference keyframe $I_r : \Omega \subset \mathbb{R}^2 \to \mathbb{R}$ and an inverse depth map $D_r : \Omega \subset \mathbb{R}^2 \to \mathbb{R}$, where Ω represents the image domain, at each timestamp. More specifically, for each pixel position $\boldsymbol{p} = (x, y)^{\mathrm{T}}$ in I_r with valid inverse depth d_p, we can obtain the corresponding projected position $\boldsymbol{p}'$ given the 3D rigid motion:

$$T = \begin{pmatrix} R & \boldsymbol{t} \\ \boldsymbol{0} & 1 \end{pmatrix} \tag{1}$$

$$\boldsymbol{p}' = \Pi\left(R\Pi^{-1}\left(\boldsymbol{p}, d_p\right) + \boldsymbol{t}\right) \tag{2}$$

This transformation matrix comprises a rotation described by an orthogonal matrix $R \in \mathrm{SO}(3)$ and a translation described by a vector $\boldsymbol{t} \in \mathbb{R}^3$. The projection function Π and its inverse Π^{-1} convert between the 2D pixel position p and the corresponding

3D point $\boldsymbol{P} = (X, Y, Z)^{\mathrm{T}}$, which can be computed as

$$\boldsymbol{p} = \Pi(\boldsymbol{P}) = \left(\frac{X}{Z} f_x + c_x, \frac{Y}{Z} f_y + c_y\right)^{\mathrm{T}} \tag{3}$$

$$\boldsymbol{P} = \Pi^{-1}(\boldsymbol{p}, d_p) = \left(\frac{x - c_x}{f_x d_p}, \frac{y - c_y}{f_y d_p}, \frac{1}{d_p}\right)^{\mathrm{T}} \tag{4}$$

where f_x and f_y are the focal lengths and c_x and c_y are the image coordinates of the principal point that compose the camera intrinsics c.

Typically, a minimal representation $\xi \in \mathfrak{se}(3)$ is used to represent the rigid motion, which is the Lie algebra associated with the Lie group $T \in \mathrm{SE}(3)$.

3.2 System architecture

Our system takes monocular image sequences as input, with optional corresponding depth images, and estimates the relative camera motion from successive frames based on the extracted edge information. The general structure of the proposed VO system is illustrated in Fig. 1; it comprises two main threads: tracking and local mapping. In the tracking thread, the main goal is to effectively calculate the 6-DoF camera pose for the incoming frames while being robust in challenging environments. To tackle this issue, we extract edge pixels for each frame, which is more robust than extracting point features in low-texture scenes, and leverage 3D–2D edge alignment to estimate the camera motion. In order to avoid sensitivity to illumination changes like direct methods, instead of using photometric residuals, we introduce the DT method to efficiently obtain geometric reprojection errors. In the local mapping thread, we focus on optimizing the depth of each edge pixel and the camera poses of selected keyframes within a sliding window. When the current frame is tracked successfully, the system determines whether the current frame is used to update the local depth map or create a new keyframe by propagating depth information from existing keyframes in the sliding window.

4 Tracking

4.1 Edge detection

For each incoming frame I_c, we aim to estimate the relative camera motion ξ_{cr} between I_c and the reference frame I_r based on edge information. We first detect edge pixels using Canny edge detection [33], which works well in challenging scenes. As shown in the left two columns of Fig. 2, the Canny algorithm is reliable when dealing with low-texture scenes, since

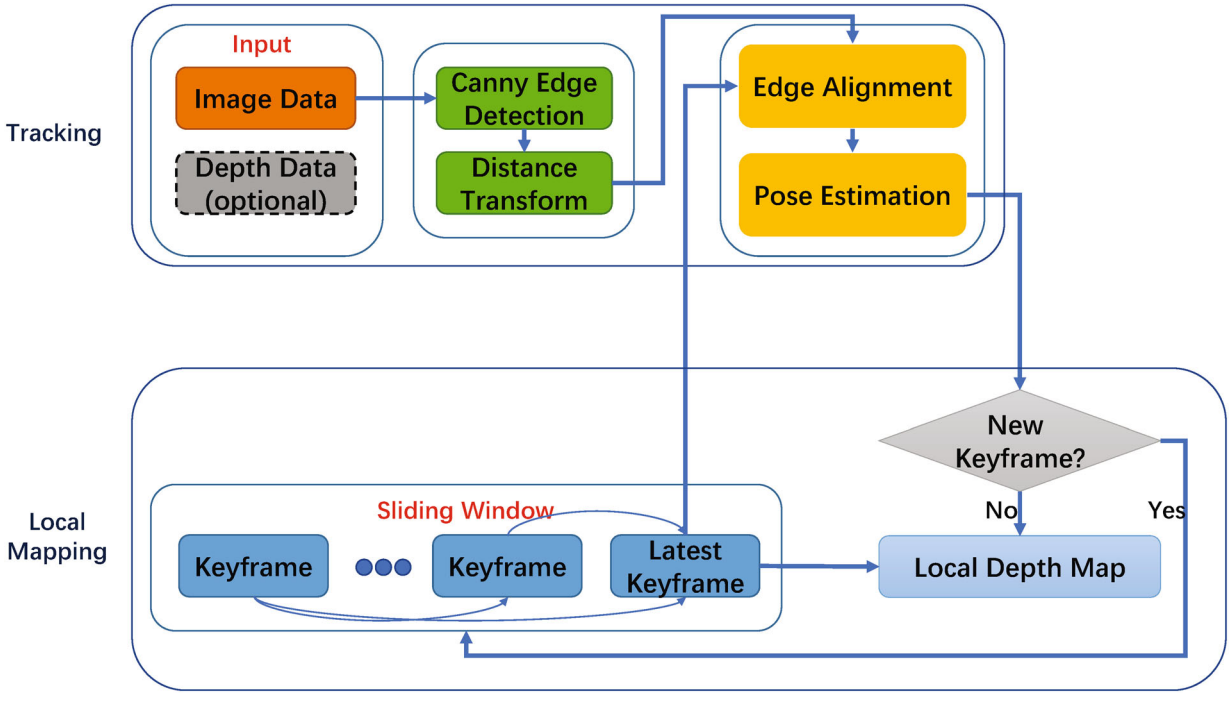


Fig. 1 System overview. Our framework comprises two main threads: tracking and local mapping. The former effectively estimates the camera pose of the current frame using 3D–2D edge alignment. The latter determines whether to add a new keyframe and optimizes it with the sliding window.

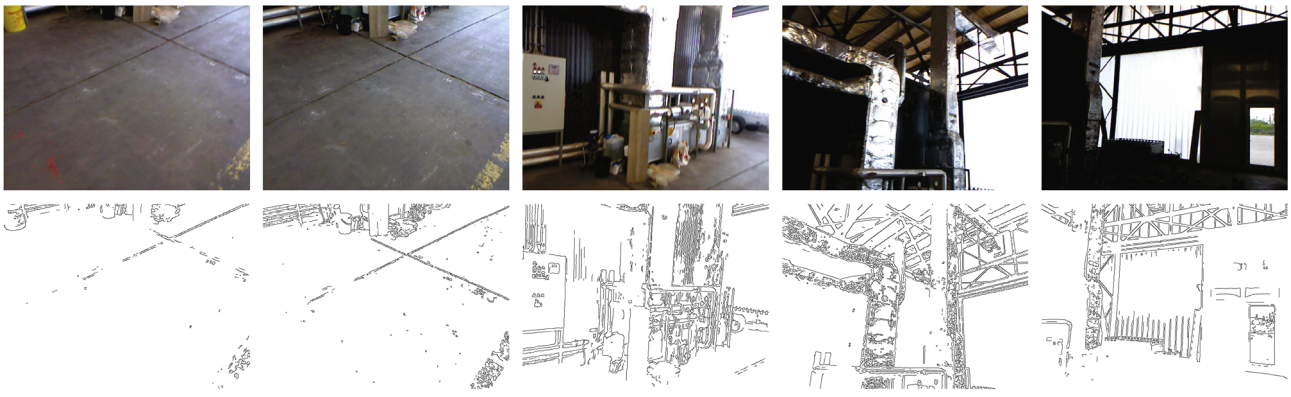


Fig. 2 Edge maps of challenging scenes. Above: original RGB images. Below: edge pixels extracted by Canny edge detection [33]. Two left columns: examples of low-texture floor. Three right columns: examples of illumination change caused by camera motion.

it locally finds the strongest edge pixels by non-maximum suppression of high gradient regions. Its further robustness to illumination changes is shown in the right three columns of Fig. 2.

4.2 Edge-based reprojection error

Typically, given the corresponding depth estimate of one edge pixel in the reference frame I_r, we can obtain its projected position (see Eq. (2)) in the current frame I_c using the initial motion estimate ξ_{cr}^0.

For computational efficiency, edge pixels are directly used to formulate the error function without extracting descriptors. Unlike direct methods that employ photometric error to track camera pose, we prefer to utilize geometric error to avoid sensitivity to illumination changes. However, without the help of features and descriptors providing place recognition capabilities, accurate projective registration of nonparametric edges is quite challenging.

Following the iterative closest point (ICP) algorithm, we utilize the distance between the projected position and the nearest detected edge pixel in the current frame to establish 3D–2D correspondence. To improve matching efficiency, we precompute the DT map [43] $D_c : \Omega \subset \mathbb{R}^2 \to \mathbb{R}$ for the current frame, which calculates the Euclidean distance to the closest edge at each pixel position. This map converts calculation of the reprojection geometric error into a simple query, and since the edge only needs to be detected once, it can be reused in the subsequent iterative optimization process without the need to repeatedly calculate the distance.

Inspired by Ref. [17], there are two main motivations for choosing geometric reprojection error instead of photometric error in this work. On the one hand, the reprojection error can alleviate sensitivity to illumination changes, and on the other hand, the non-differentiability of image intensity may have a negative effect on optimization. As shown in Fig. 3, non-differentiability of the photometric quantity is significant at object boundaries, while the DT value does not change so frequently and sharply.

More specifically, given an edge pixel e_i^r in the reference frame I_r and its inverse depth $d_{e_i^r}$, the reprojection residual is computed as

$$r(e_i^r) = D_c(e_i') \tag{5}$$

where e_i' denotes the reprojection position calculated from Eq. (2).

Since $D_c(e_i') = 0$ if e_i' is an edge pixel in I_c, we can estimate the optimal relative camera motion ξ_{cr}^* by minimizing the total error function:

$$E(\xi_{cr}) = \sum_{e_i^r \in \mathcal{E}_r} \left\| r(e_i^r) \right\|_\gamma \tag{6}$$

$$\xi_{cr}^* = \arg\min_{\xi_{cr}} E(\xi_{cr}) \tag{7}$$

where $\|\cdot\|_\gamma$ is the Huber norm and $\mathcal{E}_r$ is the set of all detected edges in I_r.

During optimization of Eq. (7), it is intuitively beneficial for the entire system if the initial motion estimate ξ_{cr}^0 is more accurate. Thus, we consider four different assumptions for ξ_{cr}^0: (i) constant motion, (ii) no motion, (iii) half- or (iv) double- that of constant motion, and choose the one with the lowest energy according to Eq. (6).

4.3 Weighted edge alignment

One of the major challenges of directly using pixels for VO systems is that it is difficult to achieve accurate data association, especially when equipped with a

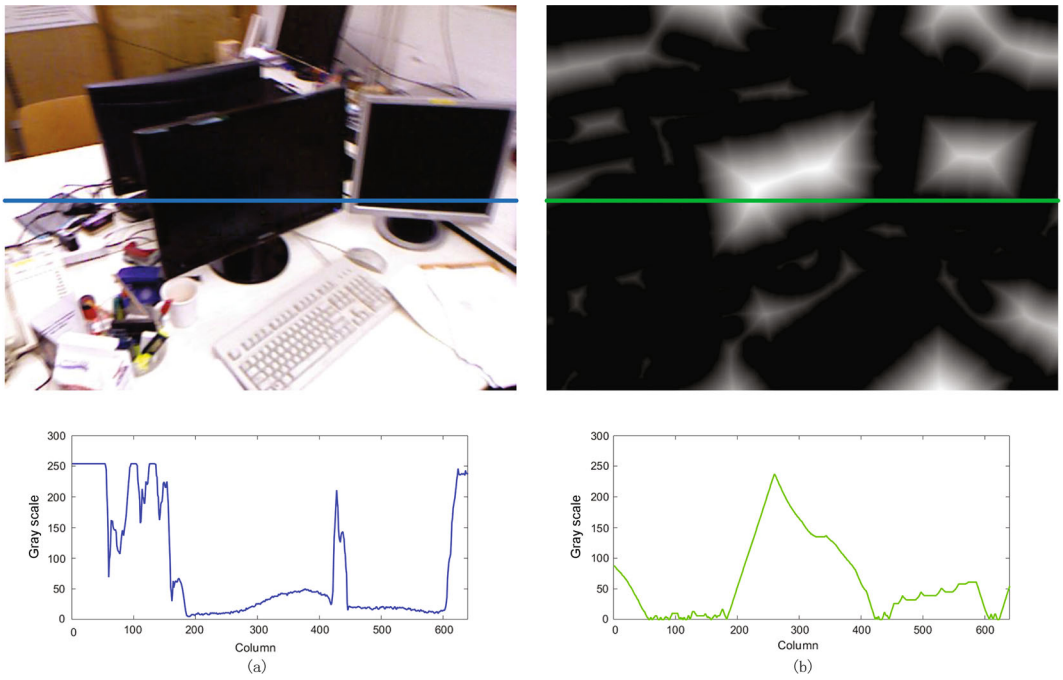


Fig. 3 Comparison of (a) photometric quantity and (b) DT value changes along a scan line (the DT value is scaled to $[0, 255]$ for visualization).

monocular camera. It is worth noting that even in direct methods [4, 11], a well-designed point selection strategy is crucial to pick locally recognizable pixels with sufficient intensity gradient for matching. In our work, however, we do not need to follow the same strategy for two reasons. On the one hand, this strategy relies on local image gradient, which is the basis of edge detection; on the other hand, the selected pixels tend to maintain a uniform spatial distribution across the whole image, but the reprojection error based on the DT map (see Eq. (5)) is unsuitable for non-edge pixels.

Thus, we propose a weighted edge alignment scheme, based on the observation that when we try to match two images like Figs. 4(a) and 4(b), more significant vertical regions can be aligned first, and then the horizontal edge with a smaller number of pixels can be used to fine-tune the alignment: see Figs. 4(c)–4(e). Intuitively, we can achieve this by gradually increasing the weight of non-significant edge regions.

More specifically, we divide edge pixels of the reference frame into 5 categories according to the local spatial layout within the 3×3 neighborhood. The first 4 main types are shown in Fig. 5, and the remaining ones that do not belong to these 4 types are classified as the 5th type. Then we count the number of edge pixels of each type and combine it with the pyramid to construct a dynamic weight that changes with the pyramid level:

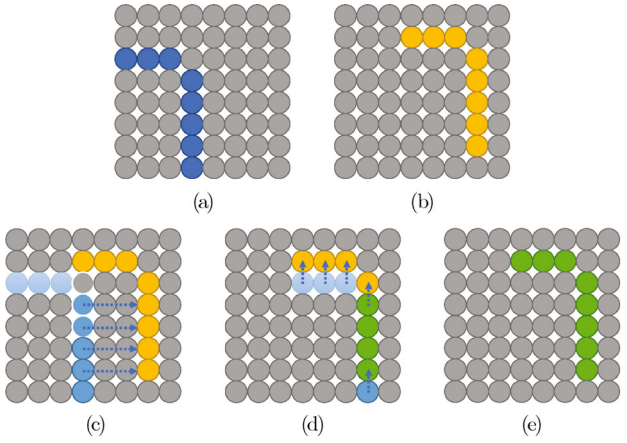


Fig. 4 Weighted edge alignment. (a, b) Edge pixels to be registered (blue and yellow, respectively). (c) Projection under a no motion assumption, and a class of edge pixels with a larger number are given a higher weight (darker colors indicate greater weights). (d) After alignment of edge pixels with larger weight, the pixel sets with a smaller number are reassigned larger weights, and matching is performed again. (e) Final matched pixels (green).

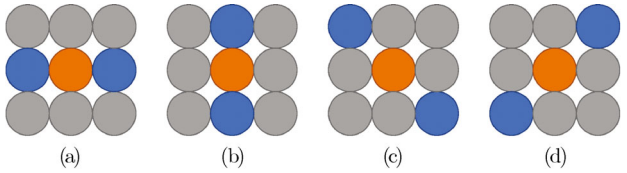


Fig. 5 4 main types of edge pixels, based on the spatial relationship between the target edge pixel (orange) and its neighboring edge pixels (blue). Non-edge pixels are in gray.

$$w_t^k = (\log(N_{\text{sum}}/N_t))^{(l-k)/l} \tag{8}$$

where $t = 0, \ldots, 4$ denotes the type, N_t is the number of edge pixels of this type, N_{sum} is the total number of edge pixels, l is the highest level of the pyramid and $k = 0, \ldots, l$ is the current level ($k = 0$ corresponds to the original image). When $k = l$, the weight of each edge pixel is 1, and as k decreases, the weight of pixel types with smaller size increases more rapidly. In practice, the weight will be within a certain range.

Now we can apply the weights to the iterative energy function calculation (Eq. (6)) in the pyramidal coarse-to-fine tracking scheme:

$$E^k(\xi_{cr}) = \sum_{e_i^r \in \mathcal{E}_r} w_{t_{e_i^r}}^k \left\| r(e_i^r) \right\|_\gamma \tag{9}$$

When at coarser pyramid levels, types with more edge pixels have higher weight and dominate registration, while at finer pyramid levels, types with fewer edge pixels receive more attention and refine the relative motion estimate. This dynamic weighting method also effectively deals with the situation where the minority follow the majority.

We further illustrate the proposed weighted edge alignment in Fig. 6. A three-level (0, 1, 2) pyramid is used to associate the extracted edge pixels of the reference frame to the edge map of the target frame. After iteration at each pyramid level, sampled edge pixels are projected into the edge map of the current frame using the estimated relative motion. Figures 6(c)–6(e) show the gradual registration of edge pixels.

4.4 Initialization for a monocular camera

When using a monocular camera, we need to initialize the first frame, which is also the first keyframe, to bootstrap the system. We follow the strategy of LSD-SLAM [4], which uses a random depth map for the first keyframe. Since the sparse edge representation offers a large convergence basin [19], the system will converge to a correct depth configuration quickly with the help of the weighted edge alignment method mentioned in Section 4.3.

4.5 RGB-D extension

The proposed VO method can be easily extended to RGB-D sensors. When the input contains depth images, the main difference from the monocular mode is in the tracking thread, and the rest of our system operates independently of the input. Since the depth sensor provides more reliable depth information, the initialization phase can be omitted for the RGB-D sensor. Furthermore, almost all detected edge pixels in the reference frame can be used in the tracking process, without the need to select edge pixels with valid depths as in the monocular mode.

5 Local mapping

In the local mapping thread, when a new keyframe is created, all edge pixels with valid depth from keyframes within a sliding window are projected into it to generate the depth map. The depths of all these edge pixels, the camera pose, and the camera parameters are optimized

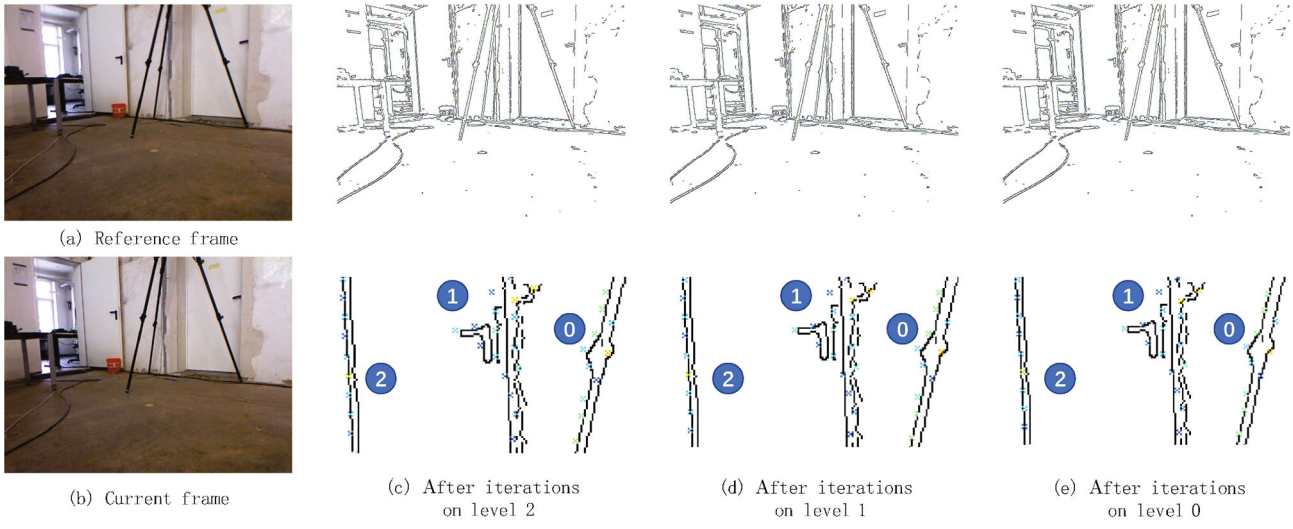


Fig. 6 Projected edge pixels after iteration, at different pyramid levels: (a) reference frame, (b) current frame, and (c–e) edge pixels projected from the reference frame to the edge map of the current frame. Above: full maps. Below: close-up. Circled numbers indicate the pixel areas mainly affected by the corresponding pyramid level.

together. Meanwhile, edge pixels and keyframes far from the current frame are marginalized.

5.1 Keyframe selection

In optimization-based VO systems, the selection of keyframes is very important to make the tracking robust to camera movement. Typically, keyframe selection ensures that a sufficient number of frames have passed since the last keyframe insertion [14], or a certain relative pose threshold has been met [4]. We also follow the same strategy and combine the edge matching method in Section 4.3 to construct three conditions as follows:

- More than 20 frames have passed since the current keyframe.
- The relative translation distance or rotation angle have met a predefined threshold.
- Among the four main edge pixel types (see Fig. 5), for at least one, the number of changes exceeds a threshold.

When one of the above conditions is met, the current frame is spawned as a new keyframe.

5.2 Sliding window optimization

We adopt a sliding window optimization method following Ref. [11]. More specifically, we keep a small sliding window $\mathcal{F}$ consisting of several (5–7) active keyframes, and jointly refine all valid edge depths, all camera poses, and even the camera intrinsics c. The residual of one edge pixel e_i of one keyframe $k_m \in \mathcal{F}$ is similar to Eq. (5):

$$r^{mn}(e_i^{k_m}) = D_{k_n}(e_i'(\xi_{wk_m}, \xi_{wk_n}, d_{e_i^{k_m}}, c)) \tag{10}$$

where e_i' is the projected position of $e_i^{k_m}$ in a different keyframe $k_n \in \mathcal{F}$, and ξ_{wk_m} and ξ_{wk_n} are the estimated camera poses relative to the world frame. Then we can minimize the final energy function in the sliding window:

$$E = \sum_{m\in\mathcal{F}} \sum_{e_i\in\mathcal{E}_m} \sum_{n\in\mathcal{F}\setminus\{m\}} \left\| r^{mn}(e_i^{k_m}) \right\|_\gamma \tag{11}$$

The energy can be optimized iteratively using Levenburg–Marquardt (L–M) algorithm:

$$\boldsymbol{H}\delta\boldsymbol{x} = -\boldsymbol{b} \tag{12}$$

where $\boldsymbol{b} = \boldsymbol{J}^{\mathrm{T}}\boldsymbol{W}\boldsymbol{r}$ consists of the Jacobian $\boldsymbol{J}$, the weight matrix $\boldsymbol{W}$, and the residual $\boldsymbol{r}$, $\boldsymbol{H} = \boldsymbol{J}^{\mathrm{T}}\boldsymbol{W}\boldsymbol{J} + \lambda\boldsymbol{I}$ is the Hessian matrix, and $\delta\boldsymbol{x}$ is the optimal increment.

Due to the bounded size of the sliding window, old keyframes need to be removed before adding new ones. Following Ref. [11], we adopt a marginalization strategy using the Schur complement. Typically, the optimization can be written in a block-matrix way:

$$\begin{bmatrix} \boldsymbol{H}_{\alpha\alpha} & \boldsymbol{H}_{\alpha\beta} \\ \boldsymbol{H}_{\beta\alpha} & \boldsymbol{H}_{\beta\beta} \end{bmatrix} \begin{bmatrix} \delta\boldsymbol{x}_\alpha \\ \delta\boldsymbol{x}_\beta \end{bmatrix} = - \begin{bmatrix} \boldsymbol{b}_\alpha \\ \boldsymbol{b}_\beta \end{bmatrix} \tag{13}$$

where α and β denote the variables that we would like to keep and to marginalize, respectively. We can eliminate the coefficient of $\delta\boldsymbol{x}_\beta$ by multiplying the second line of $\boldsymbol{H}$ by $\boldsymbol{H}_{\alpha\beta}\boldsymbol{H}_{\beta\beta}^{-1}$ and subtracting it from the first. Moreover, following Ref. [11], when marginalizing one keyframe, all edges of this keyframe and all the edges that have not been observed in the last two keyframes in the sliding window are marginalized together to retain the sparsity of $\boldsymbol{H}$.

6 Results and discussion

The proposed VO system has been implemented and tested on a desktop computer with a 3.2 GHz Intel i7-8700 CPU and 32 GB memory. Since our system permits two input modes, monocular and RGB-D, we have tested them separately in the following subsections. Two main metrics, *absolute trajectory error* (ATE) and *relative pose error* (RPE) [45] are used to respectively evaluate the global consistency of the trajectory and the drift. Meanwhile, the root mean squared error (RMSE) of the translational component is mainly used for comparison.

6.1 Monocular VO

The monocular VO was evaluated using three public datasets, including the TUM RGB-D dataset [45], the Bonn RGB-D Dynamic dataset [44], and the ICL-NUIM dataset [46]. It is worth noting that the last two datasets use the same data format as the TUM RGB-D dataset [45] and provide ground truth trajectories, so it is easy to use the evaluation tools provided by Ref. [45] for analysis.

Table 1 compares ATE and RPE for DSO [11], ORB-SLAM2 [7], DLGO [31], PL-SLAM [25], and our method. The first two methods are a direct method and a feature-based method, respectively, chosen as representatives of the most commonly used frameworks at present. The next two methods, DLGO [31] and PL-SLAM [25], are extensions of the two benchmark frameworks combining line features. The results for the first three methods are from Ref. [31]; we use the same evaluation configuration to obtain results for PL-SLAM and our method.

Table 1 Comparison of absolute trajectory RMSE (m) and relative pose RMSE (m) of DSO [11], ORB-SLAM2 [7], DLGO [31], PL-SLAM [25], and our algorithm on the TUM RGB-D dataset ("×" indicates algorithm failure)

Sequence	ATE					RPE				
	DSO	ORB	DLGO	PL-SLAM	Ours	DSO	ORB	DLGO	PL-SLAM	Ours
fr1/xyz	0.069	0.084	**0.054**	0.13	0.065	0.088	0.106	**0.067**	0.184	0.073
fr2/desk	1.65	1.207	1.33	**0.696**	0.87	1.81	1.266	1.64	**1.078**	1.088
fr2/xyz	**0.062**	0.119	0.065	0.1	0.063	**0.077**	0.109	0.081	0.142	0.079
fr3/office	1.18	1.233	1.168	1.12	**0.64**	1.512	1.577	1.464	1.586	**1.054**
fr3/cabinet	1.08	×	1.05	×	**0.84**	1.56	×	1.57	×	**1.354**
fr3/nstr_ntex_far	0.677	×	**0.504**	×	0.61	0.876	×	**0.74**	×	0.755
fr3/str_ntex_far	0.903	×	0.865	×	**0.69**	1.052	×	0.946	×	**0.80**

It can be seen that the proposed method achieves improvements in robustness and accuracy compared to the other methods. In general, our method and the two extensions perform better than the two benchmark frameworks, reflecting the effectiveness of introducing edge information to deal with low-texture environments. ORB-SLAM2 [7] and its extension fail for the last two low-texture scenes, while our method and DLGO achieve the best results respectively, showing that the use of additional image information can help to improve tracking robustness in low-texture scenes.

Since VO systems need to pay more attention to tracking, we performed a tracking status comparison on 4 sequences of the TUM RGB-D dataset (see Fig. 7). The tracking status of each frame was recorded when evaluating DSO [11], ORB-SLAM2 [7], and our method. It can be seen that ORB-SLAM2 [7] has careful initialization, but it omits many frames and has the most tracking loss. DSO [11] needs to select two frames with sufficient translational camera movement for initialization, so a few frames will be ignored at this stage. Our method directly initializes a random depth map for the first frame, and subsequent frames can be tracked successfully.

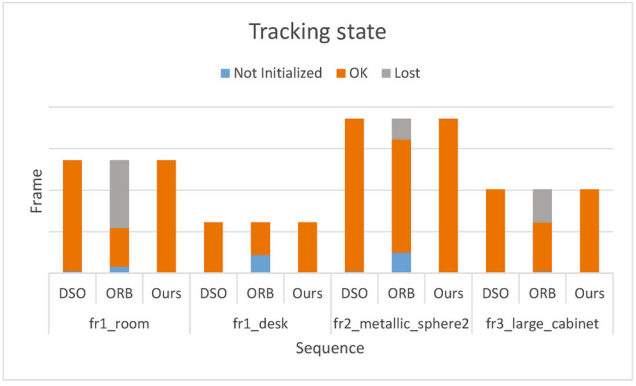


Fig. 7 Proportion of different states in the tracking process. We have tested DSO [11], ORB-SLAM2 [7], and our method on the 4 sequences of the TUM RGB-D dataset, and recorded the tracking status of each frame, including not initialized, tracking ok, and tracking lost.

The Bonn RGB-D Dynamic dataset [44] contains 24 highly dynamic sequences in which people perform different tasks, such as manipulating boxes or playing with balloons, and 2 static sequences. We chose 1 static and 6 dynamic sequences to evaluate the robustness to dynamic scenarios. There are many sudden changes in brightness of the scenes caused by camera movement in this dataset (see Fig. 8), which have a great impact on the direct method.

Table 2 compares ATE for ORB-SLAM2 [7], DSO [11], PL-SLAM [25], and our method on the 7 sequences. It is worth noting that the static sequence contains 10,916 images, which leads to a relatively large cumulative error. From Table 2, it can be concluded that our method is more robust than the other three methods in dealing with dynamic scenes



Fig. 8 Sharp illumination changes in the balloon_tracking sequence of the Bonn RGB-D Dynamic dataset [44].

Table 2 Comparison of absolute trajectory RMSE (m) of ORB-SLAM2 [7], DSO [11], PL-SLAM [25], and our algorithm on BONN-Dynamic dataset ("×" indicates algorithm failure)

Sequence	ORB	DSO	PL-SLAM	Ours
static	2.12	×	×	**1.91**
kidnapping_box	0.46	0.35	0.45	**0.29**
kidnapping_box2	0.36	0.35	**0.34**	0.39
balloon_tracking	0.31	×	0.27	**0.25**
balloon_tracking2	0.28	0.37	0.29	**0.27**
balloon	×	**0.18**	0.22	0.20
balloon2	×	×	×	**0.26**

and illumination changes.

The ICL-NUIM dataset is captured in synthetic indoor environments. Table 3 compares our method with ORB-SLAM2 [7], DSO [11], and PL-SLAM [25] in terms of ATE on the 8 office sequences, in which half have have added simulated noise. It can be seen that PL-SLAM is confused, typically because it cannot detect sufficient point and line features in untextured areas such as walls, causing it to fail. The robustness of DSO and our algorithm are better than the other two methods, and our method is competitive in terms of accuracy. Note that since ORB-SLAM2 is a complete SLAM system with re-localization and loop-closure detection, tracking loss will not directly lead to system failure, while DSO and our method directly trigger system termination if tracking is lost.

Table 3 Comparison of absolute trajectory RMSE (m) of ORB-SLAM2 [7], DSO [11], PL-SLAM [25], and our algorithm on the ICL-NUIM dataset ("×" indicates algorithm failure, and "*" indicates tracking loss occurs but does not cause failure)

Sequence	ORB	DSO	PL-SLAM	Ours
of_kt 0	0.49*	**0.26**	×	0.49
of_kt 0 (with noise)	0.61*	**0.30**	×	0.45
of_kt 1	0.83*	0.69	×	**0.64**
of_kt 1 (with noise)	0.81*	0.68	×	**0.63**
of_kt 2	**0.73**	0.79	×	0.80
of_kt 2 (with noise)	**0.75**	0.78	×	0.80
of_kt 3	0.57*	0.62	×	**0.55**
of_kt 3 (with noise)	0.59*	0.61	×	**0.53**

6.2 RGB-D VO

The VO framework proposed in this paper has good generalisability and can be integrated with depth sensors. We compare our algorithm with state-of-the-art edge-based RGB-D VO systems, including REVO [19] and CannyVO [12]. The former uses deep learning features to favor object boundaries and omit weak edges, while the latter adopts two distance transforms, approximate nearest neighbor field (ANNF) and oriented nearest neighbor field (ONNF), to improve registration in terms of efficiency and accuracy. Quantitative evaluation results are shown in Table 4. It can be seen that our method is more accurate. As shown in Fig. 9, our method can estimate a consistent camera trajectory while simultaneously recovering a semi-dense point cloud for the scene.

Figure 10 gives results of a leave-one-out ablation study using the TUM RGB-D dataset to analyse the accuracy improvements due to the proposed weighted edge alignment. We selected 8 sequences and tested each with and without the weight (see Eq. (8)) in edge alignment. We can see that the use of weighted edge alignment effectively improves accuracy.

6.3 Canny parameters

In order to compensate for the differences in scenes, we need to adapt the parameters, mainly the two thresholds $(t_{\text{high}}, t_{\text{low}})$, used by the Canny detector [33].

Figure 11 records the ATE results corresponding to different threshold selections on the 6 sequences of the TUM RGB-D dataset. It can be seen that under different scene conditions, the selection of Canny parameters has a significant impact on the results. We also give a qualitative comparison of 3D point

Table 4 Comparison of absolute trajectory RMSE (m) of REVO [19], CannyVO [12], and our method on the TUM RGB-D dataset

Sequence	REVO [19]	CannyVO (ANNF) [12]	CannyVO (ONEF) [12]	Ours
fr1/xyz	0.068	0.137	0.043	**0.033**
fr1/rpy	0.049	0.205	0.047	**0.042**
fr1/desk	0.061	0.212	**0.044**	0.054
fr1/desk2	0.082	0.381	0.187	**0.053**
fr1/room	0.298	0.621	0.242	**0.186**
fr2/desk	0.089	0.039	0.037	**0.032**
fr3/office	0.117	0.09	0.085	**0.042**

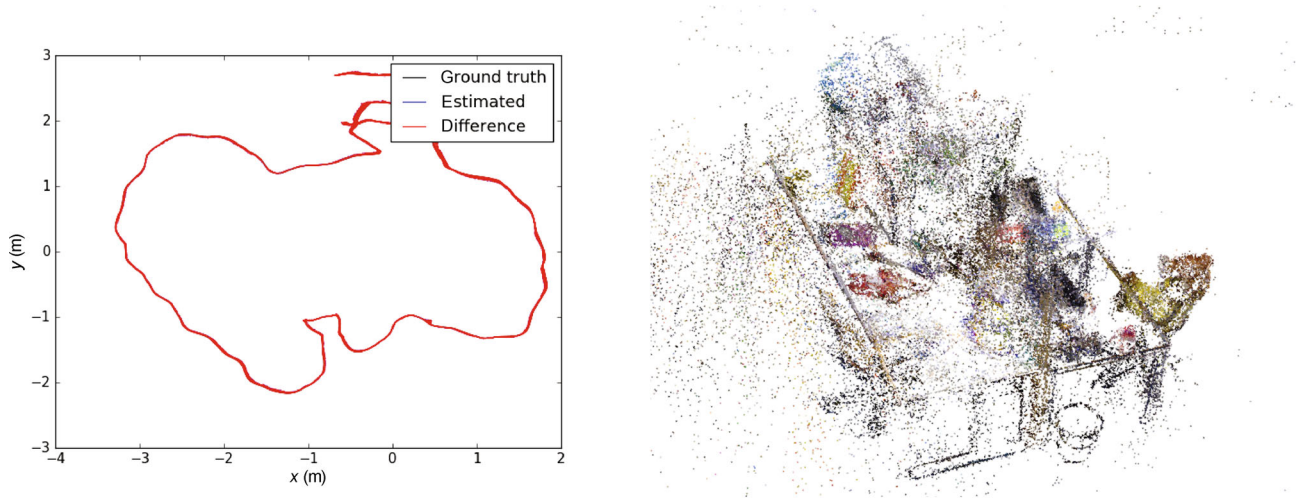


Fig. 9 Ego-motion estimation and mapping results for our proposed algorithm on the fr3/long_office sequence from the TUM RGB-D dataset.

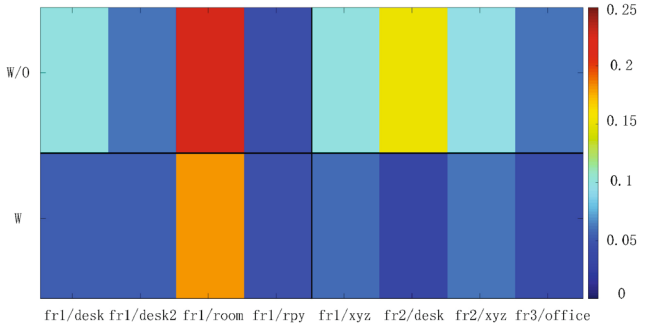


Fig. 10 Ablation study on the TUM RGB-D dataset. Each bar corresponds to the (color-coded) absolute trajectory error over the full sequence. We run each sequence (horizontal axis) without ("w/o") and with ("w") the dynamic weighted method as mentioned in Section 4.3.

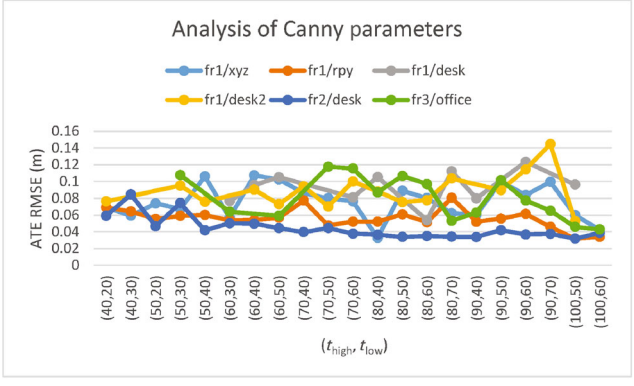


Fig. 11 Influence of different Canny parameters. We selected 6 sequences from the TUM RGB-D dataset and recorded the corresponding absolute trajectory RMSE (m) of our system in RGB-D mode using 21 different Canny parameter combinations. Results greater than 0.15 are absent.

clouds for the fr1/desk2 sequence using two different threshold combinations, $(90, 70)$ and $(100, 50)$; the influence of the Canny parameters on the generated map is obvious. If the parameters could be adjusted adaptively according to the scene, it would bring a great improvement to the proposed edge-based system, which we will considered in our future work.

6.4 Efficiency

Although there are tens of thousands of edge pixels that need to be matched in each incoming frame, with the introduction of DT, our method can still achieve excellent real-time performance. Table 5 shows the total execution time of our method for different sequences. Note that the processing efficiency is unchanged for monocular or RGB-D sensors. Table 6 compares our method with state-of-the-art edge-based and line-based VO/SLAM systems in terms of the average runtime for each module in the tracking

Table 5 Execution time of our full pipeline for different sequences ("rgbd" means the system is running in RGB-D mode)

Sequence	#Images	Time (ms)	FPS
fr2/rpy	3290	45403	72.46
fr2/xyz (rgbd)	3615	49508	73.02
lr_kt1	966	12453	77.57
of_kt0	1508	17369	86.82
balloon_tracking	590	7714	76.48

Table 6 Average execution time (ms) used by each part for per frame tracking on the TUM dataset

Operation	Yang and Scherer [30]	REVO [19]	PL-SLAM [25]	Ours
Feature extraction	28.19	7.64	31.32	2.86
Initial pose estimation	4.52	2.36	7.16	1.3
Tracking	19.23	8.53	12.58	3.46
Total	51.94	18.53	51.06	7.62



Fig. 12 Qualitative comparison of our recovered 3D maps on the fr1/desk2 sequence of the TUM RGB-D dataset using different Canny parameters.

thread. It can be concluded that our method is highly efficient and has better real-time performance than state-of-the-art methods that use edge information.

7 Conclusions

Targeting low-texture scenes which challenge existing visual odometry methods, we present a novel edge-based visual odometry algorithm that estimates the relative camera pose by minimizing the geometric reprojection error of extracted edge pixels. We experimentally demonstrate that using more image information, like direct methods, can help to deal with low-texture situations. Meanwhile, the geometric reprojection residual also makes the proposed VO system insensitive to illumination changes. Combined with a novel weighted edge alignment method, the introduced DT map further improves the accuracy and efficiency of camera tracking. Moreover, the proposed VO framework has good generality making it suitable for both monocular and RGB-D cameras. A large number of experiments conducted on public datasets show that the proposed method is comparable to state-of-the-art SLAM systems in terms of tracking accuracy, and is much faster as well as more robust in challenging scenarios.

Acknowledgements

This work was supported by the National Key R&D Program of China under Grant No. 2018YFB2100601, and the National Natural Science Foundation of China under Grant Nos. 61872024 and 61702482.

Declaration of competing interest

The authors have no competing interests to declare that are relevant to the content of this article.

Electronic Supplementary Material

Supplementary material is available in the online version of this article at `https://doi.org/10.1007/s41095-021-0251-7`.

References

[1] Blochliger, F.; Fehr, M.; Dymczyk, M.; Schneider, T.; Siegwart, R. Topomap: Topological mapping and navigation based on visual SLAM maps. In: Proceedings of the IEEE International Conference on Robotics and Automation, 3818–3825, 2018.

[2] Ling, F. F.; Elvezio, C.; Bullock, J.; Henderson, S.; Feiner, S. A hybrid RTK GNSS and SLAM outdoor augmented reality system. In: Proceedings of the IEEE Conference on Virtual Reality and 3D User Interfaces, 1044–1045, 2019.

[3] Nescher, T.; Zank, M.; Kunz, A. Simultaneous mapping and redirected walking for ad hoc free walking in virtual environments. In: Proceedings of the IEEE Virtual Reality, 239–240, 2016.

[4] Engel, J.; Schöps, T.; Cremers, D. LSD-SLAM: Large-scale direct monocular SLAM. In: *Vision – ECCV 2014. Lecture Notes in Computer Science, Vol. 8690.* Fleet, D.; Pajdla, T.; Schiele, B.; Tuytelaars, T. Eds. Springer Cham, 834–849, 2014.

[5] Engel, J.; Stückler, J.; Cremers, D. Large-scale direct SLAM with stereo cameras. In: Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems, 1935–1942, 2015.

[6] Liu, H. M.; Zhang, G. F.; Bao, H. J. Robust keyframe-based monocular SLAM for augmented reality. In: Proceedings of the IEEE International Symposium on Mixed and Augmented Reality, 1–10, 2016.

[7] Mur-Artal, R.; Tardós, J. D. ORB-SLAM2: An open-source SLAM system for monocular, stereo, and RGB-D cameras. *IEEE Transactions on Robotics* Vol. 33, No. 5, 1255–1262, 2017.

[8] Hsiao, M.; Westman, E.; Zhang, G. F.; Kaess, M. Keyframe-based dense planar SLAM. In: Proceedings of the IEEE International Conference on Robotics and Automation, 5110–5117, 2017.

[9] Engel, J.; Sturm, J.; Cremers, D. Semi-dense visual odometry for a monocular camera. In: Proceedings of the IEEE International Conference on Computer Vision, 1449–1456, 2013.

[10] Forster, C.; Zhang, Z. C.; Gassner, M.; Werlberger, M.; Scaramuzza, D. SVO: Semidirect visual odometry for monocular and multicamera systems. *IEEE Transactions on Robotics* Vol. 33, No. 2, 249–265, 2017.

[11] Engel, J.; Koltun, V.; Cremers, D. Direct sparse odometry. *IEEE Transactions on Pattern Analysis and Machine Intelligence* Vol. 40, No. 3, 611–625, 2018.

[12] Zhou, Y.; Li, H. D.; Kneip, L. Canny-VO: Visual odometry with RGB-D cameras based on geometric 3-D–2-D edge alignment. *IEEE Transactions on Robotics* Vol. 35, No. 1, 184–199, 2019.

[13] Huang, J. H.; Yang, S.; Mu, T. J.; Hu, S. M. ClusterVO: Clustering moving instances and estimating visual odometry for self and surroundings. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2165–2174, 2020.

[14] Mur-Artal, R.; Montiel, J. M. M.; Tardós, J. D. ORB-SLAM: A versatile and accurate monocular SLAM system. *IEEE Transactions on Robotics* Vol. 31, No. 5, 1147–1163, 2015.

[15] Du, Z. J.; Huang, S. S.; Mu, T. J.; Zhao, Q. H.; Martin, R.; Xu, K. Accurate dynamic SLAM using CRF-based long-term consistency. *IEEE Transactions on Visualization and Computer Graphics* https://doi.org/10.1109/TVCG.2020.3028218, 2020.

[16] Wang, X.; Dong, W.; Zhou, M. C.; Li, R. J.; Zha, H. B. Edge enhanced direct visual odometry. In: Proceedings of the British Machine Vision Conference, 35.1–35.11, 2016.

[17] Kuse, M.; Shen, S. J. Robust camera motion estimation using direct edge alignment and sub-gradient method. In: Proceedings of the IEEE International Conference on Robotics and Automation, 573–579, 2016.

[18] Schenk, F.; Fraundorfer, F. Combining edge images and depth maps for robust visual odometry. In: Proceedings of the British Machine Vision Conference, 149.1–149.12, 2017.

[19] Schenk, F.; Fraundorfer, F. Robust edge-based visual odometry using machine-learned edges. In: Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems, 1297–1304, 2017.

[20] Schenk, F.; Fraundorfer, F. RESLAM: A real-time robust edge-based SLAM system. In: Proceedings of the International Conference on Robotics and Automation, 154–160, 2019.

[21] Gee, A. P.; Mayol-Cuevas, W. Real-time model-based SLAM using line segments. In: *Advances in Visual Computing. Lecture Notes in Computer Science, Vol. 4292*. Bebis, G. et al. Eds. Springer Berlin Heidelberg, 354–363, 2006.

[22] Smith, P.; Reid, I.; Davison, A. J. Real-time monocular SLAM with straight lines. In: Proceedings of the British Machine Vision Conference, 3.1–3.10, 2006.

[23] Zhang, L. L.; Koch, R. Hand-held monocular SLAM based on line segments. In: Proceedings of the Irish Machine Vision and Image Processing Conference, 7–14, 2011.

[24] Hirose, K.; Saito, H. Fast line description for line-based SLAM. In: Proceedings of the British Machine Vision Conference, 83.1–83.11, 2012.

[25] Pumarola, A.; Vakhitov, A.; Agudo, A.; Sanfeliu, A.; Moreno-Noguer, F. PL-SLAM: Real-time monocular visual SLAM with points and lines. In: Proceedings of the IEEE International Conference on Robotics and Automation, 4503–4508, 2017.

[26] Zuo, X. X.; Xie, X. J.; Liu, Y.; Huang, G. Q. Robust visual SLAM with point and line features. In: Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems, 1775–1782, 2017.

[27] Gomez-Ojeda, R.; Briales, J.; Gonzalez-Jimenez, J. PL-SVO: Semi-direct Monocular Visual Odometry by combining points and line segments. In: Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems, 4211–4216, 2016.

[28] Gomez-Ojeda, R.; Gonzalez-Jimenez, J. Robust stereo visual odometry through a probabilistic combination of points and line segments. In: Proceedings of the IEEE International Conference on Robotics and Automation, 2521–2526, 2016.

[29] Gomez-Ojeda, R.; Moreno, F. A.; Zuñiga-Noël, D.; Scaramuzza, D.; Gonzalez-Jimenez, J. PL-SLAM: A stereo SLAM system through the combination of points and line segments. *IEEE Transactions on Robotics* Vol. 35, No. 3, 734–746, 2019.

[30] Yang, S. C.; Scherer, S. Direct monocular odometry using points and lines. In: Proceedings of the IEEE International Conference on Robotics and Automation, 3871–3877, 2017.

[31] Li, S. J.; Ren, B.; Liu, Y.; Cheng, M. M.; Frost, D.; Prisacariu, V. A. Direct line guidance odometry. In: Proceedings of the IEEE International Conference on Robotics and Automation, 5137–5143, 2018.

[32] Huang, S. S.; Ma, Z. Y.; Mu, T. J.; Fu, H. B.; Hu, S. M. Lidar-monocular visual odometry using point and line features. In: Proceedings of the IEEE International Conference on Robotics and Automation, 1091–1097, 2020.

[33] Canny, J. A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence* Vol. 8, No. 6, 679–698, 1986.

[34] Klein, G.; Murray, D. Parallel tracking and mapping for small AR workspaces. In: Proceedings of the 6th IEEE and ACM International Symposium on Mixed and Augmented Reality, 225–234, 2007.

[35] Newcombe, R. A.; Lovegrove, S. J.; Davison, A. J. DTAM: Dense tracking and mapping in real-time. In: Proceedings of the International Conference on Computer Vision, 2320–2327, 2011.

[36] Caruso, D.; Engel, J.; Cremers, D. Large-scale direct SLAM for omnidirectional cameras. In: Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems, 141–148, 2015.

[37] Wang, R.; Schwörer, M.; Cremers, D. Stereo DSO: Large-scale direct sparse visual odometry with stereo cameras. In: Proceedings of the IEEE International Conference on Computer Vision, 3923–3931, 2017.

[38] Gao, X.; Wang, R.; Demmel, N.; Cremers, D. LDSO: Direct sparse odometry with loop closure. In: Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems, 2198–2204, 2018.

[39] Lee, S. H.; Civera, J. Loosely-coupled semi-direct monocular SLAM. *IEEE Robotics and Automation Letters* Vol. 4, No. 2, 399–406, 2019.

[40] Von Gioi, R. G.; Jakubowicz, J.; Morel, J. M.; Randall, G. LSD: A line segment detector. *Image Processing on Line* Vol. 2, 35–55, 2012.

[41] Zhang, L. L.; Koch, R. An efficient and robust line segment matching approach based on LBD descriptor and pairwise geometric consistency. *Journal of Visual Communication and Image Representation* Vol. 24, No. 7, 794–805, 2013.

[42] Zhang, J.; Zeng, G.; Zha, H. Structure-aware SLAM with planes in man-made environment. In: *Computer Vision. Communications in Computer and Information Science, Vol. 773*. Yang J. et al. Eds. Springer Singapore, 477–489, 2017.

[43] Felzenszwalb, P. F.; Huttenlocher, D. P. Distance transforms of sampled functions. *Theory of Computing* Vol. 8, 415–428, 2012.

[44] Palazzolo, E.; Behley, J.; Lottes, P.; Giguère, P.; Stachniss, C. ReFusion: 3D reconstruction in dynamic environments for RGB-D cameras exploiting residuals. In: Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems, 7855–7862, 2019.

[45] Sturm, J.; Engelhard, N.; Endres, F.; Burgard, W.; Cremers, D. A benchmark for the evaluation of RGB-D SLAM systems. In: Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems, 573–580, 2012.

[46] Handa, A.; Whelan, T.; McDonald, J.; Davison, A. J. A benchmark for RGB-D visual odometry, 3D reconstruction and SLAM. In: Proceedings of the IEEE International Conference on Robotics and Automation, 1524–1531, 2014.



Feihu Yan received his Ph.D. degree in computer science from State Key Lab of Virtual Reality Technology and Systems, Beihang University, Beijing, China, in 2021. He is currently a lecturer with the School of Electrical and Information Engineering, Beijing University of Civil Engineering and Architecture, China. His current research interests include computer vision, 3D reconstruction, and SLAM.



Zhaoxin Li received his Ph.D. degree in computer application technology from Harbin Institute of Technology, China, in 2016. From September 2018 to March 2019, he worked as a postdoctoral fellow in the Department of Computing, Hong Kong Polytechnic University. He is currently with the Institute of Computing Technology, Chinese Academy of Sciences. His research interests include 3D computer vision and 3D data processing.



Zhong Zhou is a professor, Ph.D. adviser, at the State Key Lab of Virtual Reality Technology and Systems, Beihang University. He received his B.S. degree from Nanjing University and Ph.D. degree from Beihang University in 1999 and 2005, respectively. His main research interests include augmented virtual environments, natural phenomena simulation, distributed virtual environments, and Internet-based VR technologies. He is the member of IEEE, ACM, and CCF.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made.

The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

To view a copy of this licence, visit `http://creativecommons.org/licenses/by/4.0/`.

Other papers from this open access journal are available free of charge from http://www.springer.com/journal/41095. To submit a manuscript, please go to https://www.editorialmanager.com/cvmj.